

Cravitoo · Bug Fix & Security Audit Scope Questions

46 questions to ask when you brief the developer on bug hunting and code-level security auditing. These force them to commit to concrete deliverables instead of hand-wavy work.

Ask in this exact order. Anything vague = red flag.

PART A — Bug check & fix scope

1. Process & approach

Q1. How will you find bugs I don't yet know about? *(Correct answer includes: reading the codebase, running the existing test suite, manual exploratory testing, and using an automated scanner. Bad answer: "I'll just start fixing.")*

Q2. Will you deliver a written bug report before you start fixing anything? *(Insist on YES. Format: bug ID, severity, reproduction steps, screenshot, root cause, proposed fix, hours estimate.)*

Q3. What severity levels will you use — P0 / P1 / P2 / P3? Define each in plain English.

Q4. How will you prioritise which bugs to fix first — by severity, by user impact, or by fix effort?

Q5. Will you write a regression test for every bug you fix? *(Insist on YES. Bugs come back if there's no test.)*

Q6. Where will you fix each bug — preview only, or straight to production? *(Correct: preview first → I test → then production. Never straight to prod.)*

Q7. How will I verify each bug is actually fixed? *(They should give you a reproduction video, a passing test, and a preview URL you can click.)*

2. Scope & estimates

Q8. How many hours do you think it'll take to check the full codebase for bugs?

Q9. Will you charge fixed-price for the full bug sweep, or by the hour? *(Fixed-price protects you. Hourly protects them. Push for fixed with a cap.)*

Q10. What's your commitment on new bugs introduced by your fixes — free re-fix? *(Insist on a 30-day warranty on every fix.)*

Q11. Will you rewrite any modules, or only patch? *(Rewrites are dangerous. If they suggest a rewrite, ask "why" and "what's the risk if it breaks?" — a good developer will resist rewrites.)*

Q12. Which files will you touch, and which will you leave alone? *(Ask them to name a list before starting. Prevents scope creep.)*

3. Communication during the fix cycle

Q13. How often will you show me progress — daily, twice a week, weekly?

Q14. What format for status updates — Loom video, screenshots, written summary, or live call?

Q15. How will you notify me if a fix uncovers a bigger issue?

Q16. What happens if a bug turns out to be un-fixable — do I still pay? *(Correct answer: yes for time spent, but only if they document why and suggest a workaround.)*

4. Testing after fixes

Q17. Will you run the existing pytest suite before AND after each fix to catch regressions?

Q18. Will you add Playwright / end-to-end tests for the flows you touch?

Q19. What browsers / devices will you test on — Chrome desktop, Safari iOS, Chrome Android, mobile web?

Q20. How will you test payment flows without hitting live Razorpay? *(Correct: use Razorpay test-mode keys — never touch production payments during dev.)*

PART B — Code-level security audit

5. Audit scope

Q21. What does your code-level security audit cover? *(Correct answer must mention: OWASP Top 10, authentication flows, RBAC checks, secret storage, input validation, SQL/NoSQL injection, XSS, CSRF, dependency vulnerabilities, insecure defaults, hard-coded credentials, exposed debug endpoints.)*

Q22. Will you audit the backend, web frontend, AND both mobile apps — or only some? *(Insist on ALL three clients + backend.)*

Q23. Will you audit the third-party integrations — Razorpay, Resend, MongoDB, Emergent LLM key usage?

Q24. Will you audit the deployment config — env vars, CORS, cookies, TLS settings?

Q25. Will you check every API endpoint for missing authorisation checks (e.g. an employee calling a master-admin endpoint)? *(This is THE most common bug in multi-tenant apps. Push hard on this one.)*

Q26. What tools will you use for the security audit? *(Correct answer includes at least: `bandit` or `semgrep` for Python, `npm audit` and `snyc` for JS, `pip-audit` for Python deps, manual code review. Wrong answer: "I'll just eyeball it.")*

Q27. Will you check for exposed secrets in git history — not just the current code? *(Ask them to run `truffleHog` or `git-secrets` across the full history. Old commits can still leak.)*

6. Specific areas of Cravitoo I want them to check

Q28. Will you verify that `is_active=false` really terminates the session on every endpoint, not just `/auth/refresh` ?

Q29. Will you verify that the Razorpay signature verification in `/payments/razorpay/verify` cannot be bypassed?

Q30. Will you verify that a `site_admin` cannot access another site's data by tampering with `site_id` in the URL?

Q31. Will you verify that the `/auth/otp/verify` endpoint doesn't leak whether an email is registered (timing attack / different error messages)?

Q32. Will you check that the JWT secret is strong (32+ random chars) and never falls back to a default?

Q33. Will you verify that the delete endpoints (Site, Vendor, Corporate Client) can't be triggered by a lower-role user via CSRF or role escalation?

Q34. Will you check the mobile apps for insecure token storage, exposed API keys inside the APK, or debuggable release builds?

Q35. Will you scan the ~40K lines of code for hard-coded secrets, TODOs referencing security, or commented-out authentication code?

7. Deliverables from the security audit

Q36. What will the final security report look like? *(Must include: executive summary, vulnerability list with CVSS scores, affected files with line numbers, reproduction, recommended fix, effort to fix. Push for a written PDF you can archive.)*

Q37. Will you rate each finding — Critical / High / Medium / Low / Informational?

Q38. Will you fix the vulnerabilities as part of the audit fee, or is fixing separate? *(Get this in writing upfront. Ideal: Critical + High get fixed inside the audit fee. Medium + Low are quoted separately.)*

Q39. How long will the audit take — days or weeks?

Q40. Will you re-audit after fixes to confirm the vulnerability is truly closed? *(YES — a security audit without a re-audit is worthless.)*

8. What the developer must NOT do

Q41. Confirm you will NOT run any security tests against production — only against preview.

Q42. Confirm you will NOT use my real customer PII in any test — only test data.

Q43. Confirm you will NOT publish, blog about, or share ANY vulnerability finding externally, ever — even after we finish. *(Bake this into the NDA.)*

Q44. Confirm you will NOT install new tools or SaaS services that ping my data outside India without explicit written approval.

5 artefacts the developer MUST hand over at the end

If they don't deliver all five, they're not really done:

- Written **Bug Report v1** — full list of bugs found before any fixes (baseline)
- Written **Security Audit Report** with CVSS scores per finding
- Written **Fix Log** — bug ID → files changed → PR link → test coverage
- Passing **CI run** on the final `main` branch (pytest + Playwright + lint all green)
- **Post-audit changelog** — a short doc explaining what changed at a high level

Two questions to close the conversation

Q45. "What's the ONE thing you'd fix on day 1 that would make Cravitoo materially safer / more stable?" *(Great developers answer this in under 30 seconds. Bad ones ramble. Their answer also tells you their instincts.)*

Q46. "What's the ONE thing about Cravitoo's code that worries you most, based on the walkthrough today?" *(If they say "nothing looks concerning" — they didn't actually look. Walk away. Every real codebase has something worrying.)*

Warning signs during this conversation

- "I don't need to write a bug report, I'll just fix as I go"
- "Regression tests slow me down"
- "I'll audit production directly, it's fine"
- "I can do the full audit in one day" — impossible for 40K lines
- "OWASP Top 10 is outdated" — it's not, still gold standard
- They can't name a single security tool
- They ask for full production access on day one to "look around"
- They refuse fixed-price scope

Green signals

- They ask to see the code BEFORE quoting
- They suggest starting with a **1-day paid discovery** to write the bug report, then quote the fix work separately
- They mention specific tools by name (`bandit` , `semgrep` , `snyc` , `pip-audit` , `truffleHog`)
- They insist on preview → your sign-off → production for every change
- They ask for the existing test suite before writing any code
- They read your PRD.md, Bugs & Security PDF, and ask follow-up questions

Two commercial structures to propose

Option 1 — Fixed-price bug-sweep + audit package (safest for you)

"₹X for a full-codebase bug report + security audit + fix-list, delivered within 2 weeks. Critical and High findings fixed inside the fee. Medium and Low quoted separately."

Option 2 — Time & materials with a hard cap

"₹Y/hour, capped at Z hours. Weekly written status. Any hour over cap requires my written approval. All work in preview first."

Never agree to: open-ended hourly with no cap, cash-in-advance for the full amount, or "trust me, I'll be quick."